WASF299 Capstone Project: OWASP Juice Shop Security Assessment

Course: WASF299 - Capstone Project in Web Application Security
Project Duration: July 23 - August 6, 2025
Defense Date: August 6, 2025

Table of Contents

1. Project Overview
2. Assessment Methodology
3. Executive Report Template
4. Technical Report Template
5. Defense Preparation Guide
6. Appendices

Project Overview

Introduction

This capstone project requires students to conduct a comprehensive security assessment and penetration testing of the OWASP Juice Shop application. The project simulates real-world security consulting scenarios where students must identify vulnerabilities, assess risks, and provide actionable recommendations to improve the security posture of a web application.

Project Objectives

By completing this project, students will demonstrate their ability to:

1. **Conduct Professional Security Assessments**: Apply industry-standard methodologies to systematically evaluate web application security
2. **Identify and Exploit Vulnerabilities**: Discover security weaknesses using both automated tools and manual testing techniques
3. **Assess Business Risk**: Evaluate the potential impact of identified vulnerabilities on business operations
4. **Communicate Findings Effectively**: Present technical findings to both technical and executive audiences
5. **Provide Actionable Recommendations**: Develop practical remediation strategies that balance security and business requirements

Target Application: OWASP Juice Shop

Application URL: https://juice-shop.herokuapp.com

OWASP Juice Shop is a deliberately vulnerable web application designed for security training and awareness. It contains numerous security vulnerabilities that represent real-world issues commonly found in modern web applications. The application simulates an online juice shop with typical e-commerce functionality including user registration, product browsing, shopping cart management, and order processing.

Project Timeline

Duration: 2 weeks (July 23 - August 6, 2025)
Defense Date: August 6, 2025

Deliverables

1. **Executive Report** (15-20 pages): High-level summary for business stakeholders
2. **Technical Report** (30-50 pages): Detailed technical findings and recommendations
3. **Defense Presentation** (20-30 minutes): Oral presentation of findings and methodology
4. **Supporting Materials**: Evidence files, screenshots, proof-of-concept code

---

Assessment Methodology

Phase 1: Planning and Reconnaissance (Days 1-2)

1.1 Scope Definition

- Define the boundaries of the security assessment
- Identify in-scope and out-of-scope components
- Establish testing constraints and limitations
- Document rules of engagement

1.2 Information Gathering

- Application fingerprinting and technology identification
- Architecture analysis and component mapping
- User role and privilege identification
- Business logic flow documentation

1.3 Threat Modeling

- Identify potential attack vectors
- Map attack surface components
- Prioritize testing areas based on risk
- Develop testing scenarios

Phase 2: Vulnerability Discovery (Days 3-8)

2.1 Automated Scanning

- Deploy web application vulnerability scanners
- Configure scanning parameters and authentication

- Analyze automated scan results
- Validate and prioritize findings

2.2 Manual Security Testing

- Authentication and session management testing
- Input validation and injection testing
- Authorization and access control verification
- Business logic vulnerability assessment
- Client-side security evaluation

2.3 OWASP Top 10 Assessment

Systematic evaluation of each OWASP Top 10 2021 category: - A01: Broken Access Control - A02: Cryptographic Failures - A03: Injection - A04: Insecure Design - A05: Security Misconfiguration - A06: Vulnerable and Outdated Components - A07: Identification and Authentication Failures - A08: Software and Data Integrity Failures - A09: Security Logging and Monitoring Failures - A10: Server-Side Request Forgery (SSRF)

Phase 3: Exploitation and Impact Assessment (Days 9-11)

3.1 Proof of Concept Development

- Develop working exploits for identified vulnerabilities
- Document exploitation procedures and requirements
- Capture evidence of successful exploitation
- Assess the depth of compromise possible

3.2 Risk Assessment

- Evaluate likelihood and impact of each vulnerability
- Calculate risk scores using industry-standard frameworks
- Prioritize vulnerabilities based on business risk
- Consider attack chaining and escalation paths

Phase 4: Reporting and Remediation (Days 12-14)

4.1 Report Development

- Compile findings into executive and technical reports
- Develop remediation recommendations
- Create implementation timelines and priorities
- Include supporting evidence and documentation

4.2 Defense Preparation

- Prepare presentation materials
- Develop demonstration scenarios
- Anticipate questions and prepare responses
- Practice delivery and timing

Detailed Testing Methodology

Pre-Testing Setup

Environment Preparation

1. **Browser Configuration**
   - Install and configure Burp Suite or OWASP ZAP proxy
   - Configure browser to use security testing proxy
   - Disable browser security features for testing (restore after testing)
   - Install browser extensions for security testing (if applicable)
2. **Tool Installation**
   - Burp Suite Community Edition or Professional
   - OWASP ZAP (Zed Attack Proxy)
   - Nmap for network reconnaissance
   - Nikto for web server scanning
   - SQLmap for SQL injection testing
   - Custom scripts and payloads
3. **Documentation Setup**
   - Create project directory structure
   - Prepare screenshot capture tools
   - Set up evidence collection procedures
   - Initialize testing logs and notes

Initial Application Analysis

1. **Manual Exploration**
   - Browse the application without proxy interception
   - Map all visible functionality and features
   - Identify user roles and access levels
   - Document business logic flows
   - Note interesting parameters and endpoints
2. **Technology Fingerprinting**
   - Identify web server technology
   - Determine application framework
   - Analyze client-side technologies
   - Review HTTP headers and responses
   - Examine JavaScript libraries and frameworks

Phase 1: Information Gathering and Reconnaissance

1.1 Passive Information Gathering

Objective: Collect information about the target application without direct interaction

Steps: 1. **DNS and Domain Analysis** `bash nslookup juice-shop.herokuapp.com dig juice-shop.herokuapp.com whois juice-shop.herokuapp.com`

2. **Search Engine Reconnaissance**
   - Google dorking for exposed information
   - Check for cached pages and historical data
   - Search for related documentation or forums
3. **Social Media and Public Information**
   - Review OWASP Juice Shop documentation
   - Examine GitHub repository for source code insights
   - Check for known vulnerabilities and CVEs

1.2 Active Information Gathering

Objective: Directly interact with the application to gather technical information

Steps: 1. **Port Scanning** `bash nmap -sV -sC juice-shop.herokuapp.com nmap -p- juice-shop.herokuapp.com`

2. **Web Server Analysis**

   ```
   nikto -h https://juice-shop.herokuapp.com
   ```

3. **Directory and File Discovery**

   - Use tools like dirb, gobuster, or Burp Suite's content discovery
   - Look for administrative interfaces, backup files, configuration files
   - Check for common files: robots.txt, sitemap.xml, .htaccess
4. **Application Mapping**

   - Spider the application using Burp Suite or ZAP
   - Identify all endpoints and parameters
   - Map user workflows and business processes
   - Document API endpoints and AJAX calls

Phase 2: Authentication and Session Management Testing

2.1 Authentication Mechanism Analysis

Test Cases:

1. **Username Enumeration**
   - Test login form with valid/invalid usernames
   - Analyze response differences (timing, error messages, HTTP status codes)
   - Check registration form for username availability
   - Test password reset functionality for user enumeration
2. **Password Policy Testing**

- Attempt to register with weak passwords
- Test password complexity requirements
- Check for password history enforcement
- Verify account lockout mechanisms

3. **Brute Force Protection**
    - Perform automated login attempts
    - Test rate limiting and CAPTCHA implementation
    - Check for IP-based blocking
    - Verify account lockout thresholds

2.2 Session Management Testing

Test Cases:

1. **Session Token Analysis**
    - Examine session token entropy and randomness
    - Test for predictable session identifiers
    - Check session token length and complexity
    - Analyze token generation algorithms
2. **Session Lifecycle Testing**
    - Test session creation and destruction
    - Verify proper logout functionality
    - Check session timeout implementation
    - Test concurrent session handling
3. **Session Security**
    - Verify HTTPOnly and Secure flags on cookies
    - Test for session fixation vulnerabilities
    - Check for session hijacking possibilities
    - Test cross-site request forgery (CSRF) protection

Phase 3: Authorization and Access Control Testing

3.1 Horizontal Access Control

Test Cases:

1. **User Data Access**
    - Access other users' profiles or data
    - Modify user IDs in requests
    - Test for insecure direct object references
    - Check API endpoints for user data exposure
2. **Functional Access Control**
    - Access functions intended for other users
    - Test role-based access controls

- Verify user privilege boundaries
- Check for privilege escalation opportunities

3.2 Vertical Access Control

Test Cases:

1. **Administrative Function Access**
   - Attempt to access admin panels or functions
   - Test for privilege escalation to admin roles
   - Check for hidden administrative endpoints
   - Verify proper role-based restrictions
2. **Business Logic Bypass**
   - Test workflow step skipping
   - Attempt to bypass business rules
   - Check for race conditions in transactions
   - Test for price manipulation vulnerabilities

Phase 4: Input Validation and Injection Testing

4.1 SQL Injection Testing

Test Cases:

1. **Error-Based SQL Injection**
   - Test all input fields with SQL metacharacters
   - Analyze error messages for database information
   - Use single quotes, double quotes, and SQL keywords
   - Test numeric and string parameters
2. **Boolean-Based Blind SQL Injection**
   - Test for conditional responses based on SQL queries
   - Use true/false conditions to extract data
   - Test timing-based blind SQL injection
   - Automate testing with SQLmap
3. **Union-Based SQL Injection**
   - Test for UNION SELECT vulnerabilities
   - Determine number of columns in queries
   - Extract database schema and data
   - Test for file read/write capabilities

4.2 Cross-Site Scripting (XSS) Testing

Test Cases:

1. **Reflected XSS**
   - Test all input parameters for script injection

- Use various XSS payloads and encoding techniques
- Test for filter bypass methods
- Check for context-specific XSS vulnerabilities

2. **Stored XSS**
    - Test persistent input fields (comments, profiles, etc.)
    - Check for XSS in file uploads and user-generated content
    - Test for XSS in administrative interfaces
    - Verify input sanitization and output encoding
3. **DOM-Based XSS**
    - Analyze client-side JavaScript for DOM manipulation
    - Test URL fragments and hash parameters
    - Check for unsafe JavaScript functions
    - Test for client-side template injection

4.3 Other Injection Vulnerabilities

Test Cases:

1. **Command Injection**
    - Test for OS command execution in input fields
    - Use command separators and injection techniques
    - Test file upload functionality for command injection
    - Check for server-side template injection
2. **LDAP and XML Injection**
    - Test for LDAP injection in authentication forms
    - Check for XML injection in data processing
    - Test for XPath injection vulnerabilities
    - Verify input validation for structured data

Phase 5: Business Logic Testing

5.1 E-commerce Logic Testing

Test Cases:

1. **Price Manipulation**
    - Modify product prices in requests
    - Test for negative quantities or prices
    - Check for currency manipulation
    - Test discount and coupon abuse
2. **Inventory Bypass**
    - Order more items than available in stock
    - Test for race conditions in inventory management
    - Check for reservation and checkout logic flaws

- Test for bulk order restrictions

3. **Payment Processing**
   - Test for payment bypass vulnerabilities
   - Check for amount manipulation in payment flows
   - Test for refund and credit abuse
   - Verify transaction integrity

5.2 Workflow and Process Testing

Test Cases:

1. **Multi-Step Process Bypass**
   - Skip steps in registration or checkout processes
   - Test for state management vulnerabilities
   - Check for workflow validation
   - Test for process replay attacks
2. **Time-Based Logic Flaws**
   - Test for time-of-check vs time-of-use vulnerabilities
   - Check for race conditions in critical operations
   - Test for timing-based attacks
   - Verify proper synchronization mechanisms

Phase 6: Client-Side Security Testing

6.1 JavaScript Security Analysis

Test Cases:

1. **Client-Side Validation Bypass**
   - Disable JavaScript and test form submissions
   - Modify client-side validation logic
   - Test for server-side validation enforcement
   - Check for hidden form fields and parameters
2. **Sensitive Data Exposure**
   - Review JavaScript source code for hardcoded secrets
   - Check for API keys and credentials in client code
   - Test for sensitive data in local storage
   - Verify proper data handling in AJAX requests

6.2 Browser Security Features

Test Cases:

1. **Content Security Policy (CSP)**
   - Test for CSP implementation and effectiveness
   - Check for CSP bypass techniques

- Verify proper CSP configuration
- Test for inline script and style restrictions

2. **Cross-Origin Resource Sharing (CORS)**
    - Test for overly permissive CORS policies
    - Check for credential inclusion in CORS requests
    - Test for CORS misconfiguration vulnerabilities
    - Verify proper origin validation

Phase 7: Configuration and Deployment Testing

7.1 Server Configuration Testing

Test Cases:

1. **HTTP Security Headers**
    - Check for missing security headers
    - Test X-Frame-Options implementation
    - Verify X-Content-Type-Options header
    - Check for proper HSTS implementation
2. **Error Handling and Information Disclosure**
    - Test for verbose error messages
    - Check for stack traces and debug information
    - Test for directory listing vulnerabilities
    - Verify proper error page configuration

7.2 File and Directory Security

Test Cases:

1. **File Upload Security**
    - Test for unrestricted file upload vulnerabilities
    - Check file type validation and restrictions
    - Test for path traversal in file uploads
    - Verify proper file storage and access controls
2. **Path Traversal and Local File Inclusion**
    - Test for directory traversal vulnerabilities
    - Check for local file inclusion attacks
    - Test for remote file inclusion possibilities
    - Verify proper input sanitization for file paths

Documentation and Evidence Collection

During Testing

1. **Screenshot Documentation**
    - Capture screenshots of all successful exploits

- Document error messages and system responses
- Record proof-of-concept demonstrations
- Maintain organized evidence folders

2. **Request/Response Logging**
   - Save HTTP requests and responses for all tests
   - Document payload variations and results
   - Record timing information for blind attacks
   - Maintain detailed testing logs
3. **Vulnerability Documentation**
   - Document each vulnerability with clear descriptions
   - Include reproduction steps and evidence
   - Assess impact and likelihood for risk rating
   - Provide remediation recommendations

Post-Testing Analysis

1. **Risk Assessment**
   - Categorize vulnerabilities by severity (Critical, High, Medium, Low)
   - Calculate CVSS scores where applicable
   - Assess business impact and exploitability
   - Prioritize remediation efforts
2. **Report Preparation**
   - Organize findings by category and severity
   - Prepare executive summary for business stakeholders
   - Create detailed technical documentation
   - Develop remediation timelines and recommendations

Executive Report Template

Document Information

Report Title: Security Assessment of [Application Name]
Client: [Client Organization]
Assessment Period: [Start Date] - [End Date]
Report Date: [Report Date]
Prepared By: [Your Name/Team]
Classification: [Confidential/Internal Use]

Executive Summary

Assessment Overview

[Provide a high-level overview of the security assessment in 2-3 paragraphs. Explain what was tested, when it was tested, and the overall approach used. This section should be understandable by non-technical executives.]

Key Findings Summary

[Summarize the most critical findings in business terms. Focus on the potential impact to the organization rather than technical details. Use clear, concise language that executives can understand.]

Risk Assessment Summary

Risk Level	Number of Issues	Business Impact
Critical	[X]	[Brief description]
High	[X]	[Brief description]
Medium	[X]	[Brief description]
Low	[X]	[Brief description]

Recommendations Summary

[Provide 3-5 high-level recommendations that address the most critical security concerns. Focus on business value and risk reduction.]

Business Risk Analysis

Overall Security Posture

[Assess the overall security posture of the application. Discuss strengths and weaknesses from a business perspective. Consider factors such as data protection, regulatory compliance, customer trust, and operational continuity.]

Critical Business Risks

Risk 1: [Risk Name]

Business Impact: [Describe the potential business impact]
Likelihood: [High/Medium/Low]
Financial Impact: [Estimated cost or loss]
Regulatory Impact: [Compliance implications]
Reputation Impact: [Brand and customer trust implications]

Risk 2: [Risk Name]

[Follow same format as Risk 1]

Compliance and Regulatory Considerations

[Discuss any compliance requirements that may be affected by the identified vulnerabilities. Consider regulations such as GDPR, PCI DSS, HIPAA, SOX, etc.]

Strategic Recommendations

Immediate Actions (0-30 days)

[List the most critical actions that should be taken immediately to address the highest risks. Focus on quick wins and critical vulnerabilities.]

Short-term Improvements (1-6 months)

[Outline medium-term security improvements that will strengthen the overall security posture. Include process improvements and security controls.]

Long-term Security Strategy (6+ months)

[Describe strategic security initiatives that will provide long-term value and risk reduction. Consider security architecture, governance, and culture improvements.]

Resource Requirements

[Provide estimates for the resources needed to implement the recommendations, including budget, personnel, and timeline considerations.]

Implementation Roadmap

Phase 1: Critical Risk Mitigation

Timeline: [Timeframe]
Priority: Critical
Resources Required: [Description]
Success Metrics: [How success will be measured]

Phase 2: Security Enhancement

Timeline: [Timeframe]
Priority: High
Resources Required: [Description]
Success Metrics: [How success will be measured]

Phase 3: Strategic Security Improvements

Timeline: [Timeframe]
Priority: Medium
Resources Required: [Description]
Success Metrics: [How success will be measured]

Cost-Benefit Analysis

Investment Requirements

Category	Estimated Cost	Timeline
Immediate Fixes	$[Amount]	[Timeframe]
Security Tools	$[Amount]	[Timeframe]
Training	$[Amount]	[Timeframe]
Consulting	$[Amount]||[Timeframe]|| ** Total ** | **[Total]**	**[Overall Timeline]**

Risk Reduction Benefits

[Quantify the benefits of implementing the recommendations in terms of risk reduction, compliance improvement, and business value.]

Return on Investment

[Provide an analysis of the expected return on investment for the security improvements, considering both risk reduction and business benefits.]

Conclusion

[Provide a concluding statement that reinforces the importance of addressing the identified security issues and the value of the recommended improvements. Emphasize the business benefits and risk reduction that will result from implementing the recommendations.]

Next Steps

1. [Immediate action item]
2. [Follow-up action item]
3. [Long-term planning item]

Appendices

Appendix A: Methodology Overview

[Brief description of the testing methodology used]

Appendix B: Scope and Limitations

[Description of what was and was not included in the assessment]

Appendix C: Glossary of Terms

[Definitions of technical terms used in the report]

Contact Information
[Your contact information for follow-up questions]

This report contains confidential and proprietary information. Distribution should be limited to authorized personnel only.

Technical Report Template

Document Information

Report Title: Technical Security Assessment of [Application Name]
Client: [Client Organization]
Assessment Period: [Start Date] - [End Date]
Report Date: [Report Date]
Prepared By: [Your Name/Team]
Classification: [Confidential/Internal Use]
Version: [Version Number]

Table of Contents

1. Executive Summary
2. Assessment Scope and Methodology
3. Technical Findings
4. Risk Assessment
5. Remediation Recommendations
6. Appendices

Executive Summary

Assessment Overview

[Provide a technical overview of the assessment, including the scope, methodology, and key technical findings. This section should be more technical than the executive report but still accessible to technical managers.]

Key Technical Findings

[Summarize the most significant technical vulnerabilities discovered during the assessment. Focus on the technical nature of the issues and their potential for exploitation.]

Risk Summary

[Provide a technical risk assessment summary with vulnerability counts by category and severity.]

Assessment Scope and Methodology

Scope Definition

In-Scope Components

- [List all components that were included in the assessment]
- [Include URLs, IP addresses, and specific functionality tested]
- [Note any special configurations or environments]

Out-of-Scope Components

- [List any components that were explicitly excluded]
- [Explain reasons for exclusions]
- [Note any limitations or constraints]

Testing Methodology

Automated Testing

- **Tools Used**: [List all automated tools and versions]
- **Scanning Approach**: [Describe automated scanning methodology]
- **Configuration**: [Detail any special tool configurations]

Manual Testing

- **Testing Framework**: [Reference methodology used (OWASP WSTG, etc.)]
- **Testing Categories**: [List categories of manual testing performed]
- **Techniques Applied**: [Describe specific manual testing techniques]

Testing Timeline

Phase	Duration	Activities
[Phase Name]	[Duration]	[Activities Description]

Technical Findings

Vulnerability Summary

Severity	Count	Percentage
Critical	[X]	[X%]
High	[X]	[X%]
Medium	[X]	[X%]
Low	[X]	[X%]
Total	**[X]**	**100%**

Critical Vulnerabilities

CVE-[ID] - [Vulnerability Name]

Severity: Critical
CVSS Score: [Score] ([Vector])
Category: [OWASP Top 10 Category]
CWE: [CWE Number and Name]

Description: [Provide detailed technical description of the vulnerability, including the root cause and technical details of how it can be exploited.]

Location: - **URL**: [Affected URL or endpoint] - **Parameter**: [Affected parameter or component] - **Method**: [HTTP method if applicable]

Technical Details: [Provide in-depth technical analysis including:] - Root cause analysis - Code-level issues (if applicable) - Configuration problems - Architecture weaknesses

Proof of Concept:

```
[Include step-by-step reproduction steps]
1. Step one
2. Step two
3. Step three
```

Request/Response Examples:

```
[Include actual HTTP requests and responses demonstrating the vulnerability]
```

Impact Assessment: - **Confidentiality**: [High/Medium/Low] - [Explanation] - **Integrity**: [High/Medium/Low] - [Explanation] - **Availability**: [High/Medium/Low] - [Explanation]

Exploitation Complexity: [Low/Medium/High]
Attack Vector: [Network/Adjacent/Local/Physical]
Privileges Required: [None/Low/High]
User Interaction: [None/Required]

Evidence: [Include screenshots, logs, or other evidence of the vulnerability]

Remediation: [Provide specific technical remediation steps]

High Vulnerabilities

[Follow the same format as Critical Vulnerabilities for each High severity finding]

Medium Vulnerabilities

[Follow the same format for Medium severity findings, potentially with less detail]

Low Vulnerabilities

[Follow the same format for Low severity findings, with condensed information]

Risk Assessment

Risk Calculation Methodology

[Explain the risk calculation methodology used, such as CVSS v3.1, OWASP Risk Rating, or custom methodology]

Risk Matrix

Likelihood	Impact	Risk Level
High	High	Critical
High	Medium	High
Medium	High	High

Medium	Medium	Medium
Low	High	Medium
Low	Medium	Low
Low	Low	Low

Vulnerability Risk Analysis

Critical Risk Vulnerabilities

[Analyze why certain vulnerabilities are considered critical risk, including business context and exploitation scenarios]

Attack Scenarios

[Describe realistic attack scenarios that combine multiple vulnerabilities or demonstrate the potential impact of exploitation]

Compliance Impact

Regulatory Considerations

[Analyze how the identified vulnerabilities impact compliance with relevant regulations]

Standards Compliance

[Assess compliance with security standards such as OWASP Top 10, SANS Top 25, etc.]

Remediation Recommendations

Immediate Actions (Critical Priority)

Vulnerability: [Vulnerability Name]

Remediation Steps: 1. [Specific technical step] 2. [Specific technical step] 3. [Specific technical step]

Implementation Effort: [Low/Medium/High]
Timeline: [Specific timeframe]
Resources Required: [Technical resources needed]
Testing Requirements: [How to verify the fix]

Short-term Improvements (High Priority)

[Follow similar format for high priority remediation items]

Long-term Enhancements (Medium Priority)

[Follow similar format for medium priority remediation items]

Security Architecture Improvements

Input Validation Framework

[Recommend comprehensive input validation approaches]

Authentication and Authorization

[Recommend authentication and authorization improvements]

Security Monitoring

[Recommend security monitoring and logging improvements]

Secure Development Practices

[Recommend secure coding and development process improvements]

Testing Evidence

Automated Scan Results

Tool: [Tool Name]

Scan Date: [Date]
Scan Duration: [Duration]
Findings Summary: [Summary of automated findings]

[Include relevant excerpts from automated scan reports]

Manual Testing Evidence

Test Case: [Test Case Name]

Objective: [Testing objective]
Method: [Testing method used]
Result: [Test result]
Evidence: [Screenshots, logs, etc.]

Appendices

Appendix A: Vulnerability Details

[Include detailed technical information for each vulnerability, including full request/response examples, code snippets, and additional technical analysis]

Appendix B: Tool Configuration

[Document the configuration of all tools used during the assessment]

Appendix C: Testing Checklist

[Include the testing checklist used to ensure comprehensive coverage]

Appendix D: References

[List all references used in the assessment, including security standards, vulnerability databases, and technical documentation]

Appendix E: Glossary

[Define technical terms used throughout the report]

Contact Information

Primary Contact: [Name and contact information]
Technical Lead: [Name and contact information]
Project Manager: [Name and contact information]

This document contains sensitive security information and should be handled according to organizational data classification policies.

Defense Preparation Guide

Introduction

The defense presentation is a critical component of your WASF299 Capstone Project. This guide will help you prepare for a successful defense by providing structure, content recommendations, and presentation strategies. The defense is your opportunity to demonstrate your technical knowledge, analytical skills, and professional communication abilities.

Defense Format

- **Duration**: 20-30 minutes (presentation) + 10-15 minutes (Q&A)
- **Audience**: Course instructors, security professionals, and potentially other students
- **Evaluation Criteria**: Technical accuracy, methodology, findings analysis, remediation recommendations, and presentation skills
- **Delivery Method**: In-person or virtual presentation with slides and optional demonstrations

Defense Structure

1. Introduction (2-3 minutes)

- Introduce yourself and your project
- Provide context about OWASP Juice Shop and its purpose
- Explain the objectives and scope of your security assessment
- Outline the methodology and approach used

2. Assessment Methodology (3-5 minutes)

- Explain the security testing methodology used
- Discuss the tools and techniques employed
- Describe the testing environment and constraints
- Highlight any customizations or specific approaches

3. Key Findings (8-10 minutes)

- Present the most significant vulnerabilities discovered
- Categorize findings by severity and impact
- Provide clear examples of critical vulnerabilities
- Demonstrate exploitation techniques (if appropriate)
- Connect findings to OWASP Top 10 or other security frameworks

4. Risk Assessment (3-5 minutes)

- Explain your risk assessment methodology
- Present the business impact of identified vulnerabilities
- Discuss potential attack scenarios and threat actors
- Analyze compliance implications

5. Remediation Recommendations (3-5 minutes)

- Present prioritized remediation strategies
- Explain technical solutions for critical vulnerabilities
- Discuss strategic security improvements
- Provide implementation timelines and resource requirements

6. Conclusion (1-2 minutes)

- Summarize key points and lessons learned
- Highlight the value of the security assessment
- Discuss potential future work or additional testing

7. Q&A Session (10-15 minutes)

- Prepare for questions about your methodology, findings, and recommendations
- Be ready to defend your technical decisions and risk assessments
- Consider preparing backup slides for potential questions

Presentation Best Practices

Content Preparation

- **Focus on Significance**: Emphasize the most critical findings and their impact
- **Technical Depth**: Demonstrate technical understanding without overwhelming the audience
- **Visual Evidence**: Include screenshots, code snippets, and diagrams to support findings

- **Narrative Flow**: Create a logical progression from methodology to findings to recommendations
- **Time Management**: Rehearse to ensure you can cover all material within the time limit

Slide Design

- **Clarity**: Use clear, concise language and avoid technical jargon when possible
- **Visual Appeal**: Use consistent design elements and appropriate color schemes
- **Information Density**: Limit text on slides; use visuals to convey complex information
- **Accessibility**: Ensure text is readable and colors have sufficient contrast
- **Professional Appearance**: Use a consistent template and proper formatting

Delivery Techniques

- **Practice**: Rehearse your presentation multiple times to build confidence
- **Pacing**: Speak clearly and at a moderate pace
- **Engagement**: Maintain eye contact and engage with the audience
- **Technical Precision**: Use correct terminology and be precise in your explanations
- **Confidence**: Project confidence in your findings and recommendations
- **Adaptability**: Be prepared to adjust your presentation based on audience feedback

Demonstration Guidelines

If you plan to include live demonstrations of vulnerabilities or exploitation techniques:

- **Preparation**: Ensure your demonstration environment is properly configured
- **Backup Plan**: Have screenshots or videos ready in case of technical issues
- **Ethical Considerations**: Emphasize the ethical context of your testing
- **Clarity**: Explain each step of the demonstration clearly
- **Impact**: Connect the demonstration to real-world impact
- **Time Management**: Keep demonstrations concise and focused

Handling Questions

- **Listen Carefully**: Ensure you understand the question before responding
- **Concise Responses**: Provide clear, focused answers
- **Technical Accuracy**: Maintain technical precision in your responses
- **Honesty**: If you don't know an answer, acknowledge it and offer to follow up
- **Professionalism**: Respond professionally to challenging questions
- **Preparation**: Anticipate potential questions and prepare responses

Common Defense Questions

Prepare for these common questions that may arise during your defense:

1. **Methodology Justification**:
    - "Why did you choose this particular testing methodology?"

  - "How did you prioritize your testing efforts?"
  - "What were the limitations of your approach?"
2. **Finding Validation**:
  - "How did you verify that this vulnerability is exploitable?"
  - "What evidence supports your severity rating for this finding?"
  - "Could this vulnerability be a false positive?"
3. **Risk Assessment**:
  - "How did you calculate the risk scores for your findings?"
  - "Why is this vulnerability rated as critical rather than high?"
  - "What business impacts could result from this vulnerability?"
4. **Remediation Effectiveness**:
  - "How would your recommended fix address the root cause?"
  - "Are there alternative remediation approaches?"
  - "How would you verify that the remediation is effective?"
5. **Technical Understanding**:
  - "Can you explain the technical details of how this vulnerability works?"
  - "What security controls would have prevented this vulnerability?"
  - "How does this vulnerability relate to the OWASP Top 10?"
6. **Professional Judgment**:
  - "What was the most challenging aspect of this assessment?"
  - "If you had more time, what additional testing would you perform?"
  - "How would you approach this assessment differently in the future?"

Defense Preparation Checklist

Use this checklist to ensure you're fully prepared for your defense:

- ☐ Create a professional slide deck following the recommended structure
- ☐ Prepare clear explanations for all critical and high-risk findings
- ☐ Develop concise demonstrations or visual evidence of key vulnerabilities
- ☐ Practice your presentation to ensure it fits within the time limit
- ☐ Prepare for potential questions and objections
- ☐ Test all technical demonstrations in advance
- ☐ Review your executive and technical reports for consistency with your presentation
- ☐ Arrange for any necessary equipment or technical setup
- ☐ Dress professionally and arrive early (for in-person defenses)
- ☐ Test your audio, video, and screen sharing (for virtual defenses)

Evaluation Criteria

Your defense will be evaluated based on the following criteria:

Technical Content (50%)

- Accuracy and depth of technical findings
- Appropriateness of testing methodology
- Quality of vulnerability analysis
- Effectiveness of remediation recommendations
- Comprehensiveness of risk assessment

Presentation Quality (30%)

- Organization and structure
- Clarity of communication
- Visual presentation quality
- Time management
- Professionalism

Q&A Response (20%)

- Accuracy of responses
- Depth of technical knowledge
- Professional communication
- Critical thinking and problem-solving
- Adaptability to questions

Final Tips

- **Know Your Audience**: Adjust technical depth based on the audience's expertise
- **Tell a Story**: Create a narrative that connects vulnerabilities to real-world impact
- **Be Authentic**: Acknowledge challenges and limitations in your assessment
- **Show Enthusiasm**: Demonstrate passion for security and your findings
- **Professional Demeanor**: Maintain professionalism throughout the defense
- **Prepare Thoroughly**: Comprehensive preparation builds confidence

Remember that the defense is not just about presenting findings but demonstrating your growth as a security professional. Use this opportunity to showcase your technical skills, analytical thinking, and professional communication abilities.

Appendices

Appendix A: OWASP Top 10 2021 Reference

A01:2021 - Broken Access Control

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits.

A02:2021 - Cryptographic Failures

Failures related to cryptography (or lack thereof) often lead to sensitive data exposure or system compromise. Previously known as Sensitive Data Exposure.

A03:2021 - Injection

Injection flaws, such as SQL, NoSQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization.

A04:2021 - Insecure Design

Insecure design refers to risks related to design and architectural flaws, with a call for more use of threat modeling, secure design patterns, and reference architectures.

A05:2021 - Security Misconfiguration

Security misconfiguration is the most commonly seen issue. This is commonly a result of insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information.

A06:2021 - Vulnerable and Outdated Components

Components, such as libraries, frameworks, and other software modules, run with the same privileges as the application. If a vulnerable component is exploited, such an attack can facilitate serious data loss or server takeover.

A07:2021 - Identification and Authentication Failures

Application functions related to authentication and session management are often implemented incorrectly, allowing attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users' identities temporarily or permanently.

A08:2021 - Software and Data Integrity Failures

Software and data integrity failures relate to code and infrastructure that does not protect against integrity violations. This can happen when applications rely on plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs).

A09:2021 - Security Logging and Monitoring Failures

This category helps detect, escalate, and respond to active breaches. Without logging and monitoring, breaches cannot be detected. Insufficient logging, detection, monitoring, and active response occurs any time: - Auditable events, such as logins, failed logins, and high-value transactions, are not logged. - Warnings and errors generate no, inadequate, or unclear log messages. - Logs of applications and APIs are not monitored for suspicious

activity. - Logs are only stored locally. - Appropriate alerting thresholds and response escalation processes are not in place or effective. - Penetration testing and scans by dynamic application security testing (DAST) tools do not trigger alerts. - The application cannot detect, escalate, or alert for active attacks in real-time or near real-time.

A10:2021 - Server-Side Request Forgery (SSRF)

SSRF flaws occur whenever a web application is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).

Appendix B: Recommended Tools

Vulnerability Scanners

- **OWASP ZAP**: Free and open-source web application security scanner
- **Burp Suite**: Comprehensive web application security testing platform (Community and Professional editions)
- **Nikto**: Web server scanner for dangerous files and outdated server software
- **Acunetix**: Automated web vulnerability scanner

Proxy Tools

- **Burp Suite Proxy**: Intercept and modify HTTP/HTTPS traffic
- **OWASP ZAP Proxy**: Alternative to Burp Suite with similar functionality
- **Fiddler**: Web debugging proxy for any browser, system, or platform

Network Tools

- **Nmap**: Network discovery and security auditing
- **Wireshark**: Network protocol analyzer
- **Netcat**: Networking utility for reading from and writing to network connections

Exploitation Tools

- **SQLmap**: Automated SQL injection and database takeover tool
- **Metasploit Framework**: Penetration testing framework
- **BeEF**: Browser Exploitation Framework for testing browser vulnerabilities

Password Tools

- **Hydra**: Online password cracking tool
- **John the Ripper**: Offline password cracker
- **Hashcat**: Advanced password recovery utility

Mobile Testing

- **MobSF**: Mobile Security Framework for automated mobile app security assessment
- **Drozer**: Android security assessment framework

API Testing

- **Postman**: API development and testing platform
- **SoapUI**: API testing tool for SOAP and REST APIs

Code Analysis

- **SonarQube**: Static code analysis platform
- **OWASP Dependency-Check**: Software composition analysis tool
- **Bandit**: Python-specific static code analyzer

Documentation Tools

- **Markdown editors**: For report writing and documentation
- **Screenshot tools**: For capturing evidence
- **Greenshot**: Screenshot software with annotation capabilities
- **OWASP Threat Dragon**: Threat modeling tool

Appendix C: Useful Resources

OWASP Resources

- OWASP Web Security Testing Guide
- OWASP Top 10
- OWASP Juice Shop Project
- OWASP Juice Shop GitHub Repository

Security Standards and Frameworks

- NIST Cybersecurity Framework
- ISO 27001
- PTES - Penetration Testing Execution Standard
- OSSTMM - Open Source Security Testing Methodology Manual

Vulnerability Databases

- CVE - Common Vulnerabilities and Exposures
- CWE - Common Weakness Enumeration
- CVSS - Common Vulnerability Scoring System
- NVD - National Vulnerability Database

Learning Resources

- PortSwigger Web Security Academy
- HackTheBox
- TryHackMe
- SANS Courses

Books

- "The Web Application Hacker's Handbook" by Dafydd Stuttard and Marcus Pinto
- "OWASP Testing Guide" by OWASP Foundation

- “Web Penetration Testing with Kali Linux” by Juned Ahmed Ansari
- “Real-World Bug Hunting” by Peter Yaworski

Appendix D: Glossary of Terms

- **Authentication**: The process of verifying the identity of a user, system, or entity
- **Authorization**: The process of determining whether an authenticated entity has permission to access a resource or perform an action
- **CSRF (Cross-Site Request Forgery)**: An attack that forces authenticated users to submit unwanted requests to web applications
- **CVSS (Common Vulnerability Scoring System)**: A framework for communicating the characteristics and severity of software vulnerabilities
- **CWE (Common Weakness Enumeration)**: A community-developed list of common software and hardware weakness types
- **Exploit**: A piece of software, data, or sequence of commands that takes advantage of a vulnerability to cause unintended behavior
- **IDOR (Insecure Direct Object Reference)**: A vulnerability where an application provides direct access to objects based on user-supplied input
- **JWT (JSON Web Token)**: A compact, URL-safe means of representing claims to be transferred between two parties
- **OWASP (Open Web Application Security Project)**: A nonprofit foundation that works to improve the security of software
- **Penetration Testing**: A simulated cyber attack against a computer system to check for exploitable vulnerabilities
- **SAST (Static Application Security Testing)**: A testing methodology that analyzes source code to find security vulnerabilities
- **DAST (Dynamic Application Security Testing)**: A testing methodology that analyzes running applications to find security vulnerabilities
- **SQL Injection**: A code injection technique used to attack data-driven applications by inserting malicious SQL statements
- **Vulnerability**: A weakness in a system that can be exploited by a threat actor
- **XSS (Cross-Site Scripting)**: A type of injection attack where malicious scripts are injected into trusted websites